

Performance Enhancements and General Fixes

GSoC 2019 Proposal

Kanav Gupta
kanav0610@gmail.com

April 2019

Abstract

DifferentialEquations.jl is the state of the art differential equations solver that exists right now. Being written purely in Julia and having a huge number of algorithms - this package provides both complete and easy interface for users. Being said that, to maintain this state of the art performance, we need to keep updating the code and always need to have a right check in terms of performance. This project aims to do the same. The project aims to - reduce the memory consumption of the solvers, increase the speed (step optimizations), increased number of options for optional optimizations and improved benchmarking techniques.

1 Description of the Project

Ordinary Differential Equations (ODEs) are an important part of science. They occur at very frequently in varied fields of science. But most of the time the analytical solution is not available/doesn't exist. There, numerical solvers come into play. DifferentialEquations.jl is one of the most feature rich and most optimized package used for solving differential equations. Performance had always been top priority for this package. It is highly optimized towards memory and speeds. It also packs some optional optimizations for some special cases so that people with specific needs may also have option to make it more optimized. For example, we can set dense option to false to not save extra data required for interpolations thus having an option for people to save extra memory. Here, I propose some new ideas to further enhance the benchmarks of this package and also introduce some features which allow users to optionally increase the performance by removing other features.

1.1 Subproject 1: Refactoring of Codebase

Currently, the whole code base have some inconsistencies in terms of both design and style. There are many places in the code where we have done some sort of workaround (just to make it work). The code also looks a bit daunting for new contributors. But the main concern of all this is the huge compilation time on the first run (i.e. JIT compilation). For example a simple `ODEProblem` with function `f(du,u,p,t) = du .= u` takes this much time and memory on the first run:

```
julia> @time solve(prob, Tsit5())
4.618552 seconds (12.69 M allocations: 648.389 MiB...
```

This is something which is more of a mainstream Julia problem, but we can try to restructure our code that may help in reducing these metrics from benchmarks.

Also current structure requires special implementation of `perform_step!` for each algorithm. This is quite unnecessary theoretically, as it only loops through all the constants from the butcher tableau, but some algorithms need some special attention like different error estimators. Also we need to make sure they work well with GPUs. Thus we need a common `perform_step!` implementation for a given class of algorithms. This would help us delete a ton lot of code. Implementation of this would require a lot of macroprogramming and maybe a custom implementation of broadcast (to make it GPU ready). A very basic implementation is already implemented in `ExplicitRK` method which goes like -

```
for i = 2:stages-1
    utilde = zero(kk[1])
    for j = 1:i-1
        utilde = utilde + A[j,i]*kk[j]
    end
    kk[i] = f(uprev + dt*utilde,p,t+c[i]*dt)
end
```

This would help us reduce the work in the future too. For example, when `DEStats` was implemented, we had to go through every implementation of `perform_step!` and add required number of increments. Having a common routine can reduce these changes to a single line! But we need to make sure this works well for other algorithms as well as we need GPU support.

Also for algorithms like `SDIRK`, a table is enough i.e. `perform_step!` is not required. We can remove them, given we are taking care of the above mentioned peculiarities.

All such optimizations need to be taken care of in this sub project. All

this must be done before any other subproject because this may change the structure of `OrdinaryDiffEq.jl` by a great amount so it would be better if all other changes may require them to be done twice.

1.2 Subproject 2: Reaching Theoretical Limits of memory for Algorithm Caches

Certain limitations (maybe in the structure) are stopping us from reaching the theoretical limit of memory consumption. But as it was seen in #691 , wrappers may be the solution to this problem.

To give an example, suppose we want to calculate -

$$S1 := S1 + F(S2)$$

The naive approach would be to do the following -

$$\begin{aligned} S3 &= F(S2) \\ S1 &= S1 + S3 \end{aligned}$$

Problem with this is that we need an extra register $S3$.

The solution to this was to make a wrapper which contains reference to $S1$ and overload assignment (or to be precise, `copyto!`) with `+=` and pass this wrapper to the output array i.e. `du`. This means that when we do

$$\text{du} .= \text{u}$$

We are actually doing -

$$\text{du} .+= \text{u}$$

Then we pass this wrapper to the user-provided function. This will not change anything for user, would only help us to decrease one extra register.

This approach was used in and was already implemented through Williamson-Wrapper. But this approach is limited to Low memory methods for now. There may be many places in the code where such wrappers can help reduce the cache size significantly!

Also we currently don't have 2R methods available in low memory methods. These methods assume that we can do transformation of form $S = F(S)$ without need of extra register. This can be done for functions like $F(\vec{x}) = A\vec{x}$ where A is upper triangular matrix. Current structure of `ODEProblem` don't allow such optimization and these need to be implemented as well.

The goal is to check for all the possibilities of such optimizations (at least optional optimization) and implement the same wherever possible.

1.3 Subproject 3: API for preallocated Caches

Currently, solving a problem allocates memory for cache every time `solve()` is called. If the user needs to solve a problem many times using the same algorithm, this leads to unnecessary repeated allocation of memory for cache.

The idea is to make an exposed API endpoint for `alg_cache()` (internal function in `OrdinaryDiffEq` which allocates memory for algorithm cache) function so that whenever user knows that he/she may need to run the ‘solve’ routine many time, the cache may be saved

```
myCache = cache_api(Tsit5())
```

And then pass the cache to the `solve` or `init` interface:

```
sol = solve(prob, Tsit5(),...; cache = myCache)
```

1.4 Subproject 4: Approximate Matrix Factorization Techniques

To explain the idea lets take an example of Semilinear ODE. These are ODEs of form -

$$y' = Ly + f(y, t) \quad (1)$$

where L is a linear operator and f is a nonlinear equation.

Fully implicit schemes solve at each step a linear system where the matrix involves the Jacobian of the right hand side function. Efficient schemes specialized in the solution of (1) such as linearly implicit methods use only the part of the Jacobian associated with the stiff linear term, resulting in linear systems of the form

$$(1 - h\gamma L)x = l \quad (2)$$

where h is the step size, γ is a parameter of the integration scheme, and the right-hand side l is determined by the method.

The matrix L is usually sparse but has a large bandwidth, especially when high order discretization schemes are applied. Consequently, the LU factorization of the matrix $(1 - \gamma h L)$ can be very costly for huge scale problems.

Approximate Matrix Factorization is a technique to increase the computational efficiency of the solvers for these type of problems. Assuming that the matrix is a sum of simpler matrices

$$L = \sum_{r=1}^R L_r$$

AMF replaces the system matrix with a product of simpler, and easier to factorize, matrices

$$1 - h\gamma L \approx \prod_{r=1}^R (1 - h\gamma L_r)$$

This method not only works for semilinear ODEs but also for general nonlinear splitting ODEs. All we need is to get this split from the user (somewhat in the lines of `SplitODEProblem`). Many articles use this approach in specific algorithms to speed up the solver routine [4] [1]. We already have the required articles, all we need is to implement them. Idea is to implement multiple factorization techniques and have a keyword called `factorization` in specific algorithms (like currently we have keyword `nl_solve` to decide the nonlinear solver method for the given algorithm) with default value set to `lu()` (no optimization) but have an option `amf()` too for this optimization.

```
struct MyAlgo{F} <: OrdinaryDiffEqAlgorithm
    # other entries
    factorization::F
end

sol = solve(prob, MyAlgo(factorization=amf()), ...) # With AMF
sol = solve(prob, MyAlgo(factorization=lu()), ...)  # Without AMF
```

This will also help us to further extend with new factorization techniques by just defining the methods and passing it to the algorithm!

1.5 Subproject 5: Implementation of New Time Stepping Algorithms

Adaptive stepsize methods are an important part of `DifferentialEquations.jl`. These methods change their step-size ‘dt’ according to the accuracy required for the problem. *Many small steps should tiptoe through treacherous terrain, while a few great strides should speed through smooth uninteresting countryside*

What we actually do is we check if the local truncation error is within our accuracy bounds. If it is, then we try to increase the step-size for the next step as long as the error is in limits. But if it is not, we fail the step taken and retry the step with smaller dt . Controllers are the function which decides the step size for the next step using the error of the previous step(s) and the current dt .

Currently, we have several step-size controllers - standard, predictive and PI controller. New controllers like PID controller may bring about more

accurate and optional accuracy to the package! Papers for such (or using such) controllers are also being published

1.6 Stretch Goals

1.6.1 Quantum Solvers for Differential Equations

I am very interested in Quantum Computing and Algorithms and am very interested in implementing algorithms for solving differential equations. There is ongoing work in JuliaDiffEq in this using Yao.jl and would contribute to it if I manage to complete my project before the deadline.

1.7 Potential Mentors

Christopher Rackauckas (@ChrisRackauckas) and Yingbo Ma (@YingboMa) are going to mentor me for this project during the summers.

2 Timeline

Buffer Period: April 9 — May 6 I would also try to complete my remaining PRs or tasks (if any) and try to finish them before the community bonding period. I will try to go through Hairer’s Book’s [2] [3] important topics.

Community Bonding: May 6 — May 27 I plan to get to know more people in the Julia Community during this time period. I will need to brush up any theory that may be required during the coding phase. Also I would start the my first sub-project of Refactoring the Codebase during this period.

Coding Phase Starts, Subproject 1.1: May 27 — June 14 Continue doing the subproject 1.1 (Refactoring the code base). Would release improved benchmarks with the community.

Subproject 1.3: June 14 — June 24 Completing the Subproject: API for Preallocated Caches

First Evaluation: June 24 — June 28 Two sub-projects to be presented for evaluation.

Subproject 1.2: June 24 — July 10 Trying to reach theoretical limits for different algorithms, especially low memory methods. Making a sub module for all such wrappers.

Subproject 1.5: July 10 — July 27 Implementation of New Time Stepping Algorithms. Second Evaluation also lie in this period, So I plan to implement a couple of these before that.

Subproject 1.4: July 27 — August 19 Implementation of New Approximate Matrix Factorization Techniques.

3 Previous Contributions

3.1 Major Contributions

- Developed `WilliamsonWrapper` which lets the Package reach the theoretical limit of cache size for the low storage 2N methods:
<https://github.com/JuliaDiffEq/OrdinaryDiffEq.jl/pull/691>
- Implements Anderson Acceleration prototype for solving non-linear equations:
<https://github.com/JuliaDiffEq/OrdinaryDiffEq.jl/pull/580>
<https://github.com/JuliaDiffEq/OrdinaryDiffEq.jl/pull/605>
<https://github.com/JuliaDiffEq/OrdinaryDiffEq.jl/pull/630>
- Worked towards development of `DEStats` to generate statistics related to solving differential equations:
<https://github.com/JuliaDiffEq/OrdinaryDiffEq.jl/pull/670>
<https://github.com/JuliaDiffEq/OrdinaryDiffEq.jl/pull/671>
<https://github.com/JuliaDiffEq/OrdinaryDiffEq.jl/pull/678>
<https://github.com/JuliaDiffEq/OrdinaryDiffEq.jl/pull/696>
<https://github.com/JuliaDiffEq/DiffEqBase.jl/pull/193>
<https://github.com/JuliaDiffEq/DiffEqBase.jl/pull/197>

3.2 Minor Contributions

- Cache size reduction in `Verns`:
<https://github.com/JuliaDiffEq/OrdinaryDiffEq.jl/pull/570>
<https://github.com/JuliaDiffEq/OrdinaryDiffEq.jl/pull/572>
<https://github.com/JuliaDiffEq/OrdinaryDiffEq.jl/pull/650>

- Documentation Fixes:
<https://github.com/JuliaDiffEq/DiffEqDocs.jl/pull/188>
<https://github.com/JuliaDiffEq/DiffEqDocs.jl/pull/212>
- Interpolation Fix in SimpleATsit5:
<https://github.com/JuliaDiffEq/SimpleDiffEq.jl/pull/20>

4 Deliverables

I plan to present the results in the form of improved benchmarks shared using blogs and notebooks. A comprehensive suite for benchmarking already exists in DiffEqDevTools.jl. Deliverables for the sub-projects would be:

- **Refactoring the codebase** - Decreased compilation time for generic problems.
- **Reaching theoretical limits for Algorithm Caches** - Optional decreased size of caches for the user (through some new keyword arguments)
- **API for preallocated caches** - Endpoint to create and pass caches
- **Approximate Matrix Factorization Techniques** - Create an optional AMF routine like `lu` to be passed for factorization method.
- **Implementation of new time stepping algorithms** - New controllers made available for users.

5 About Me

I am a computer science undergraduate (sophomore) at Indian Institute of Technology, Roorkee. I am very much interested in Cryptography and all aspects of Mathematics. I am also interested in Quantum Algorithms and find this new field really cool. I got introduced to Numerical Methods last semester and later got interested in it. Then I joined the JuliaDiffEq community and started out doing little contributions to it. I dedicate my motivation to work here to the awesome community which is always ready to help out beginners.

I am part of SDS Labs, a student group at IIT Roorkee that constantly tries to innovate and foster technical activities in campus. We explore wide range of fields and interests related to computer science and try to make innovative products.

5.1 Academic Details

- University: Indian Institute of Technology, Roorkee
- Major: Computer Science
- CGPA: 9.0

5.2 Contact Information

Email: kanav0610@gmail.com

GitHub: kanav99

6 Summer Logistics

I would be able to work more than 35 hours a week, I don't have any other commitments or part-time jobs in the summer. Just my college reopens around 12 July so time I could devote during weekdays may be reduced but I can still manage 4 hours on weekdays and 6 hours on weekends.

References

- [1] S. GONZALEZ-PINTO, D. HERNANDEZ-ABREU, AND S. PREZ-RODRIGUEZ, *Rosenbrock-type methods with inexact amf for the time integration of advection-diffusion-reaction pdes*, Journal of Computational and Applied Mathematics, 262 (2014), pp. 304 – 321. Selected Papers from NUMDIFF-13.
- [2] E. HAIRER, S. P. NØRSETT, AND G. WANNER, *Solving Ordinary Differential Equations I (2Nd Revised. Ed.): Nonstiff Problems*, Springer-Verlag, Berlin, Heidelberg, 1993.
- [3] E. HAIRER AND G. WANNER, *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*, Springer Series in Computational Mathematics, Springer Berlin Heidelberg, 2010.
- [4] H. ZHANG, A. SANDU, AND P. TRANQUILLI, *Application of approximate matrix factorization to high order linearly implicit runge-kutta methods*, CoRR, abs/1408.3622 (2014).